Memorial Descritivo Técnico — T2 (Kart Multiplayer)

Oct 27, 2026 · @Matheus

Identificação do projeto

O T2 evolui o jogo de kart multiplayer do T1: dois jogadores correm 2 voltas numa pista com um NPC fiscal animado que patrulha a lateral e comemora o fim da corrida.

Item	Valor
Engine	Unreal Engine 5.8, somente Blueprints
Perspectiva	Terceira pessoa (câmera atrás do kart)
Rede	Listen Server, 2 jogadores, lógica de regras no servidor
Mapas	`MainMap` (lobby) → `NewMap` (corrida de 2 voltas, via `ServerTravel`)
Disciplina	Pós-Graduação em Design de Jogos (PUCRS) — Profª Rossana B. Queiroz
Integrantes	Matheus Cassali e Angelo Almeida

A.1 Técnicas de animação

O projeto usa duas técnicas: animação esquelética no NPC fiscal e cinemática procedural por física no kart.

NPC fiscal (esqueleto + AnimBP). Malha `SKM_Manny_Simple` no esqueleto `SK_Mannequin`, controlada pelo `ABP_NPC_Fiscal` (AnimInstance). A State Machine `SM_Locomocao` tem quatro estados:

Estado	Conteúdo	Transição de saída
Locomocao	Blend Space 1D `BS1D_NPC_Locomocao` (Idle 0 → Walk 150 → Jog 500, eixo Velocidade)	`EstaNoAr` → Pulo

Estado	Conteúdo	Transição de saída
Pulo	`MM_Jump` (sem loop)	automática no fim da animação → Queda
Queda	`MM_Fall_Loop` (loop)	`NOT EstaNoAr` → Aterrissagem
Aterrissagem	`MM_Land` (sem loop)	automática → Locomocao

O salto é a ação contextual: quando a corrida termina, o fiscal para a patrulha e pula de alegria a cada 1,5 s.

Kart (física). O kart não tem esqueleto. O movimento vem de forças aplicadas ao corpo rígido (`Simulate Physics`), com traçados de linha nas rodas; a inércia e a suspensão resultam da simulação, não de clipes.

Game feel e legibilidade. O Blend Space dá transição contínua entre parado, andando e correndo, sem estalos. O pulo comunica o fim da corrida em todas as telas, mesmo para quem não está olhando o HUD.

A.2 Extração de dados para a State Machine

O Event Graph do `ABP_NPC_Fiscal` lê duas variáveis do personagem a cada frame e a State Machine só consulta essas variáveis.

1. `Event Blueprint Initialize Animation` → `Try Get Pawn Owner` → `Cast To Character` → guarda a referência em `Personagem`.
2. `Event Blueprint Update Animation` → `Is Valid (Personagem)`:
   - `Velocidade` = `Get Velocity` → `Vector Length XY` (ignora o eixo vertical);
   - `EstaNoAr` = `Character Movement` → `Is Falling`.
3. `Velocidade` alimenta o eixo do Blend Space; `EstaNoAr` decide as transições Locomocao → Pulo e Queda → Aterrissagem.

Em rede. A IA roda só no servidor; o `Character Movement` replica posição e modo de movimento. Cada máquina calcula `Velocidade` e `EstaNoAr` localmente a partir do movimento replicado, então a animação não precisa ser replicada.

Anim Notify. O notify `Passo` foi marcado nos quadros de contato do pé em `MF_Unarmed_Walk_Fwd` e `MF_Unarmed_Jog_Fwd` (guiado pelos Sync Markers L/R). No AnimBP, `AnimNotify_Passo` toca `SW_Passo` com `Play Sound at Location` e atenuação `ATT_Passo` (falloff 1500). O Blend Space usa *Notify Trigger Mode = Highest Weighted Animation* para não disparar passos duplicados na mistura.

B.3 Estruturação do cenário

A pista é feita de peças estáticas modulares e de poucos atores compostos que concentram a lógica de corrida.

- **Geometria modular:** chão e paredes como Static Meshes e blocos de *Modeling Tools* (pasta `_GENERATED`), reaproveitados ao longo da pista; as paredes servem de bloqueio de colisão.
- **`BP_TrackSpline`:** ator com um `SplineComponent` que descreve o traçado. O kart consulta a spline para saber se cruzou a linha no sentido certo (produto escalar entre a direção da spline e a velocidade).
- **`BP_FinishLine`:** ator com um `BoxComponent` de overlap que conta as voltas por kart, só no servidor.
- **`BP_BotaoLobby`:** gatilho no `MainMap` que leva os jogadores ao `NewMap` via `ServerTravel`.
- **World Partition:** o `NewMap` salva cada ator num arquivo separado (*One File Per Actor*), o que facilita o versionamento em Git e carrega só as células necessárias.

Benefício. Peças repetidas compartilham mesh e material, o que reduz memória e draw calls. A lógica fica em poucos Blueprints nomeados (`BP_`, `GM_`, `W_`), fáceis de achar no Content Browser.

B.4 Necessidade e uso de NavMesh

Sim: o NPC fiscal é o agente que consome a NavMesh, patrulhando a lateral da pista perto da linha de chegada.

- **Limites navegáveis:** um `NavMeshBoundsVolume` no `NewMap` cobre só a faixa lateral, fora do traçado dos karts, atravessando o piso. A área verde (tecla P) confirma a geração.
- **Agente:** `BP_NPC_Fiscal` (Character, Max Walk Speed 180, *Orient Rotation to Movement*) possuído por `BP_AIC_Fiscal` (AIController, *Auto Possess AI = Placed in World or Spawned*).
- **Comportamento:** no `On Possess`, o controller chama `IrParaProximoPonto`, que usa `AI MoveTo` até o próximo `Target Point` da lista `PontosPatrulha` (Instance Editable). Ao chegar, espera 2 s e avança o índice com módulo; em falha, tenta de novo após 1 s.
- **Interrupção:** quando a corrida termina, `GM_Kart` chama `Comemorar` em cada fiscal: `Stop Movement`, `Comemorando = true` e a patrulha não é retomada.
- **Rede:** o AIController só existe no servidor, que resolve o caminho; os clientes recebem o movimento replicado do Character.

B.5 Papel da câmera

A câmera em terceira pessoa fica atrás e acima do kart para o jogador ler a curva seguinte e a posição do adversário ao mesmo tempo.

Parâmetro (SpringArm do `BP_KartPhysics`)	Valor	Função
Target Arm Length	500	distância da câmera ao kart
Socket Offset (Z)	100	eleva a câmera para enxergar a pista acima do kart
Enable Camera Lag / Camera Lag Speed	ativado / 6	suaviza a posição em acelerações e batidas
Enable Camera Rotation Lag / Rotation Lag Speed	ativado / 4	suaviza o giro nas curvas, mais lento que a posição
Do Collision Test / Probe Size / Probe Channel	ativado / 12 / Camera	uma esfera de raio 12 testa a linha até o kart e aproxima a câmera quando uma parede fica no caminho
Use Pawn Control Rotation	desativado	a câmera segue a rotação do kart (Inherit Pitch/Yaw/Roll), não a do mouse

A câmera e o mapeamento de input só são ativados no kart controlado localmente (`Is Locally Controlled`), então cada jogador vê apenas o próprio kart.

C.6 Elementos obrigatórios de interface

Três elementos cobrem o estado da partida: início, progresso e fim.

Elemento	Widget	Quando aparece	O que informa
Contagem regressiva	`W_Contagem` (dentro de `W_HUD`)	ao entrar no `NewMap` com 2 jogadores	“3, 2, 1, JÁ!”; os karts ficam travados até o “JÁ!”
Contador de voltas	`W_LapCount` (dentro de `W_HUD`)	durante toda a corrida	volta atual do jogador local “X/2”, limitado a 2 com `Min`

Elemento	Widget	Quando aparece	O que informa
Tela de fim de jogo	`W_EndGameUI`	quando o primeiro kart completa 2 voltas	“FIM DO JOGO” e “Vencedor: Jogador N”; o input é desativado e o cursor aparece

Os textos ficam centralizados com âncora no centro e alinhamento 0,5/0,5, em cor contrastante com o céu claro da pista.

C.7 Referências e inspirações

A interface segue o padrão das duas maiores referências de kart arcade, *Mario Kart* e *Crash Team Racing*: pouca informação na tela, números grandes e mensagens de estado no centro.

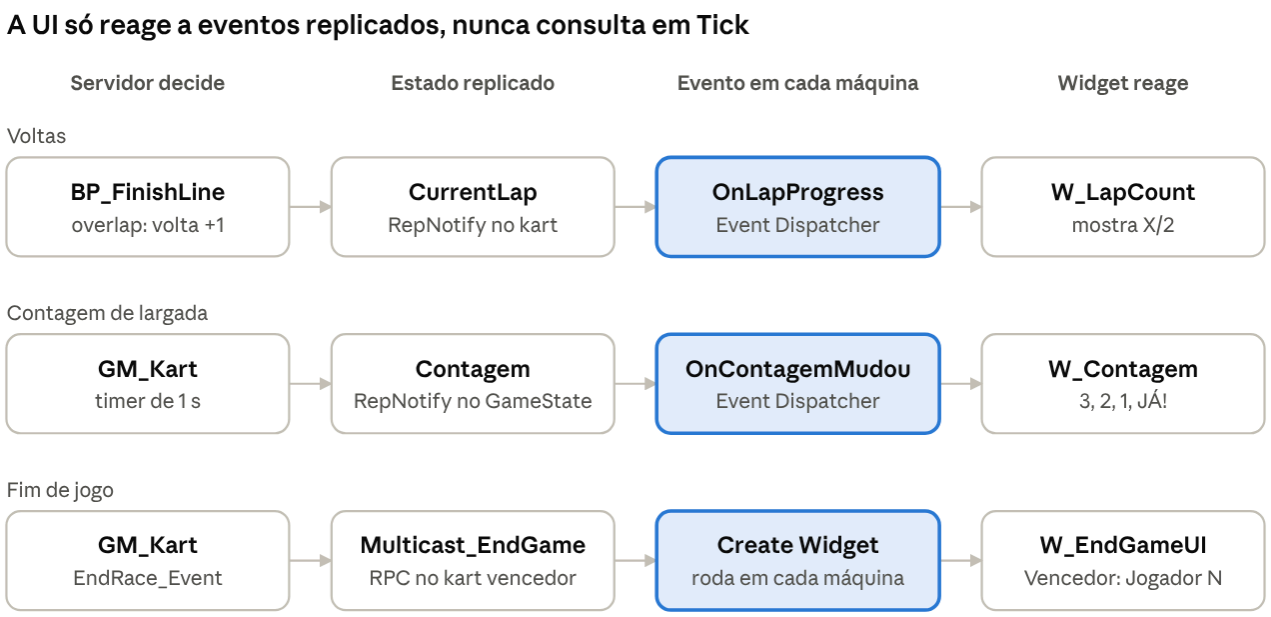
- **Contador de voltas no canto inferior** (“X/2”), como em *Mario Kart* — consultado de relance sem tirar o olho da pista.
- **Contagem “3, 2, 1, JÁ!” grande e central**, padrão de largada do gênero — todos sabem exatamente quando podem acelerar.
- **Tela de fim centralizada com o vencedor**, que interrompe o controle — não deixa dúvida de que a partida acabou.

Essa disposição atende à clareza porque o centro da tela só é ocupado nas transições de estado; durante a corrida ele fica livre para a pista.

Por serem os jogos mais conhecidos do gênero, o jogador já reconhece essa disposição e não precisa aprender a ler o HUD.

C.8 Comunicação ator-interface

Nenhum widget consulta dados em Tick ou Property Binding: o servidor altera um estado replicado, e cada máquina avisa seus widgets por um evento.



fluxo dos dados até a UI · 3 casos

- **RepNotify** (`CurrentLap` no kart, `Contagem` no `BP_GS_Kart`): a variável replica do servidor e a função `OnRep_` roda em cada máquina quando o valor muda.
- **Event Dispatcher** (`OnLapProgress` , `OnContagemMudou`): o `OnRep_` só dispara o evento; o widget se inscreve com `Bind Event` no `Construct` . O ator não conhece o widget, e o widget só conhece o evento.
- **Multicast RPC** (`Multicast_EndGame`): para um momento único como o fim de jogo, o servidor chama o evento no kart vencedor e cada máquina cria a tela localmente.
- **GameState** guarda o estado que todos veem (`Contagem` , `CorridaComecou`); o **GameMode**, que só existe no servidor, guarda as regras e os timers.

D.9 Simulação física e colisão

Sim: o kart é um corpo rígido simulado, e sem a física não há direção, derrapagem nem batida — a mecânica central do jogo.

- **Kart:** `BoxComponent` com `Simulate Physics` ; aceleração e direção chegam ao servidor por `Server_SetAcceleration` e `Server_SetSteer` , que aplicam as forças. Durante a contagem, um `Select` troca a aceleração por 0 enquanto `CorridaComecou` for falso.
- **Linha de chegada:** detecção por overlap (`ActorBeginOverlap` + `Cast To BP_KartPhysics`), sem bloqueio físico; só o servidor (`Has Authority` do kart) conta a volta.
- **NPC:** cápsula do Character, movida pelo `Character Movement` sobre a NavMesh.

Ator	Perfil (Object Type)	Collision Enabled	Responde a
Kart (`BP_KartPhysics` , Box)	`Kart` (canal de objeto próprio `Kart`)	Query and Physics	bloqueia tudo: chão, paredes, outros karts, NPC; gera overlap na linha de chegada
Linha de chegada (`BP_FinishLine` , Box)	`LinhaChegada` (WorldDynamic)	Query Only	ignora todos os canais e faz overlap só com `Kart`
Paredes e chão	`BlockAll` (padrão)	Query and Physics	bloqueiam kart, NPC e a sonda da câmera
NPC fiscal	`Pawn` (padrão do Character)	Query and Physics	bloqueia chão, paredes e karts

O canal `Kart` foi criado com resposta padrão *Block*, então tudo que já existia no mapa continua colidindo com o kart. Como `LinhaChegada` só enxerga `Kart` , o NPC, a câmera e outros objetos atravessam a linha sem disparar o overlap que conta voltas.

D.10 Especificação do MVP

O MVP é uma corrida de 2 voltas para 2 jogadores em rede, do lobby à tela de vencedor, com um NPC animado e navegável no cenário.

Pilar	Base indispensável entregue
Animação	`ABP_NPC_Fiscal` com State Machine de 4 estados, Blend Space 1D por velocidade, Anim Notify `Passo` com som atenuado, salto contextual no fim da corrida
Navegação	`NavMeshBoundsVolume` + `BP_AIC_Fiscal` patrulhando Target Points com `AI MoveTo`; câmera third-person com SpringArm e lag
Interface	HUD com contador de voltas, contagem regressiva e tela de fim, atualizados por eventos (RepNotify + Event Dispatcher), sem Tick nem Binding
Rede (base do T1)	Listen Server, regras no servidor (`GM_Kart`), estado compartilhado em `BP_GS_Kart`, voltas por kart com RepNotify, Multicast para o fim de jogo

Fora do MVP: mais de 2 jogadores, ranking de posições, botão de voltar ao lobby e pausa.